

Credit Card Fraud Detection

MSDS 422 Final Project Report

Andrea Scherben, Shelagh Haney, Ryenn McAdory | Northwestern University | MSDS 422

Executive Summary

Card-not-present and card-present fraud continue to erode issuer margins and customer trust. This project developed a machine-learning pipeline to flag fraudulent credit card transactions at the moment they occur, giving the business a data-driven way to intercept losses before they settle rather than discovering them after the fact.

We worked with a real-world dataset of 284,807 transactions made by European cardholders over a two-day window in September 2013, of which only 492 (0.173%) are confirmed fraud. This extreme imbalance was the central business challenge: a model that simply predicted “not fraud” every time would be 99.8% accurate and 0% useful, so our evaluation approach was built around metrics that reward actually catching fraud — recall, precision, and area under the precision-recall curve — rather than raw accuracy.

We sourced and validated the dataset, completed exploratory data analysis, and carried out data preparation and feature engineering, including a stratified train/test split that preserved the true fraud rate in both partitions. No records were dropped for missing values, duplication, or outlier status, since each of those, in this dataset, plausibly reflects genuine transaction behavior rather than data-quality errors.

Modeling results: we built and compared four classifiers — Logistic Regression, K-Nearest Neighbors, Linear Discriminant Analysis, and Quadratic Discriminant Analysis. K-Nearest Neighbors delivered the best overall balance of precision and recall (PR-AUC of 0.841), correctly identifying over 80% of fraud cases in testing while keeping false alarms low. After tuning the decision threshold to reflect the actual cost of a missed fraud versus a false alarm, the model caught nearly 90% of fraud cases in our test set. Scaled to the full dataset, that performance translates to an estimated net benefit in the tens of thousands of dollars over just the two-day window this dataset covers — detail and caveats are in the Findings section below.

We recommend the business move forward with a KNN-based (or, for lower-latency real-time scoring, an LDA-based) fraud-scoring model, paired with a threshold set by ongoing cost analysis rather than a fixed default, and a phased deployment plan described in this report.

Problem Statement / Research Objectives

Business problem: undetected fraudulent transactions create direct financial losses for issuers, chargeback and operational costs, and reputational damage. Conversely, overly aggressive fraud rules create false declines that frustrate legitimate customers and cost the business sales. The organization needs a model-based approach that catches a materially higher share of fraud than current rules while keeping false positives at a manageable level.

Research objectives:

- Determine which transaction-level features are most predictive of fraud, and whether engineered features (e.g., time-of-day) add predictive value beyond the anonymized PCA components.
- Build and compare at least four supervised classification models suited to severe class imbalance.
- Identify the evaluation metrics and decision threshold that best balance fraud capture (recall) against false-positive rate, given the asymmetric cost of the two error types.

- Translate model performance into an estimated business impact (potential fraud losses avoided vs. review/labor cost of false positives).
- Recommend a practical deployment and automation strategy for scoring transactions in near real time.

Unit of analysis: a single credit card transaction. Target variable: Class (1 = confirmed fraud, 0 = legitimate). Key assumption: the 28 principal components (V1–V28) were derived by the data provider via PCA on the original, undisclosed transaction features for confidentiality, so the modeling features are already de-identified and decorrelated by construction; only Time and Amount remained in their raw, interpretable form.

Exploratory Data Analysis

The dataset contains 284,807 transactions recorded over a 48-hour window, with 30 predictors (V1–V28, Time, and Amount) and a binary target, Class. Only 492 transactions (0.173%) are labeled fraud — a ratio of roughly one fraudulent transaction per 579 legitimate ones.

Fraud vs. legitimate transaction amounts

Fraudulent transactions average \$122.21 versus \$88.29 for legitimate ones — but the median tells a different story: the typical fraudulent transaction (\$9.25) is smaller than the typical legitimate one (\$22.00). Over half (51%) of fraudulent transactions were under \$10, and the single largest fraudulent transaction was \$2,125.87. This pattern — many small-dollar fraud attempts alongside a few large ones — is consistent with attackers testing stolen card numbers with small charges before attempting larger ones, and it argues against using transaction amounts alone as a screening rule.

Missing values, duplicates, and outliers

No missing values were found anywhere in the dataset. 1,081 duplicate rows were identified (1,062 legitimate, 19 fraud). We chose to retain all duplicates and outliers rather than drop them: in a transaction log, a repeated row is plausibly a genuine repeated charge (or, in the case of fraud, a genuine repeated attack pattern) rather than a data-entry error, and removing them would risk deleting real fraud signal.

Feature correlations

Because V1–V28 are anonymized PCA components, we cannot interpret them as named business features, but correlation with the Class label still identifies which components carry the strongest fraud signal. Several V-features showed materially stronger positive or negative correlation with fraud than the rest, giving the models a concentrated set of informative predictors rather than 28 equally weak ones (see the correlation chart in the appendix notebook for the full ranking).

Data Preparation / Feature Engineering

- **Feature engineering:** the raw Time field (seconds elapsed since the first transaction) was converted into Hour_of_Day, a cyclical 0–24 feature intended to capture time-of-day fraud patterns in a form models can use directly.
- **Assumption testing:** distribution analysis confirmed a right-skew in Amount, and the correlation heatmap confirmed low pairwise linear correlation among most V1–V28 features (i.e., limited multicollinearity).
- **Scaling, deferred deliberately:** V1–V28 arrive already centered and scaled from the provider's PCA transform. We deferred scaling Time/Amount until the modeling stage itself, so the scaler could be fit strictly on training data — avoiding any leakage of test-set statistics into the transform.

- **Train/test split:** an 80/20 stratified split (227,845 training rows, 56,962 test rows) preserved the 0.173% fraud rate in both partitions, so no partition over- or under-represents the minority class.

Methodology and Tools

Tooling and environment

The pipeline was developed in Python 3 within a notebook environment (Jupyter/Google Colab, based on the gdown-based data retrieval step), using pandas and NumPy for data handling, scikit-learn for modeling and evaluation, and Matplotlib/Seaborn for visualization. All four models are classical machine learning algorithms rather than deep learning, so training ran entirely on CPU — no GPU or TPU acceleration was required.

Evaluation approach

Given the 0.173% fraud rate, accuracy is not a meaningful metric — a model predicting “not fraud” for every transaction scores 99.8% accuracy while catching zero fraud. We instead evaluated every model on precision, recall, F1, ROC-AUC, and PR-AUC (average precision), with PR-AUC as the primary metric for model selection, since it is the measure least distorted by severe class imbalance.

Models implemented

- **Logistic Regression (baseline):** a linear classifier trained with `class_weight='balanced'` so the minority fraud class is not drowned out by the majority class in the loss function. Regularization strength was tuned via a small grid search ($C \in \{0.01, 0.1, 1, 10\}$) scored on average precision.
- **K-Nearest Neighbors:** a non-parametric, distance-weighted classifier with no distributional assumptions — included as a non-linear contrast to the other three models.
- **Linear Discriminant Analysis (LDA):** assumes each class is multivariate normal with a shared covariance matrix. Our own assumption testing found Amount to be right-skewed and did not confirm multivariate normality across the V-features, so LDA was included deliberately as a test of how much that assumption violation costs in practice, rather than assumed to fail outright.
- **Quadratic Discriminant Analysis (QDA):** relaxes LDA's shared-covariance assumption, allowing each class its own covariance structure — a reasonable next step given fraud and legitimate transactions plausibly have different variance structures across the anonymized features.

Key decisions

- **Class-imbalance handling:** class weighting over resampling (SMOTE/undersampling) — this preserves every real transaction in training rather than discarding data or synthesizing fraud examples that may not reflect real attack patterns.
- **Threshold as a business lever:** the standard 0.5 cutoff was used only to train and initially compare models. The deployed decision threshold was separately tuned to maximize an estimated net dollar benefit (see Findings), since 0.5 is an arbitrary default with no connection to the actual cost asymmetry between missed fraud and false alarms.
- **Small-sample caveat:** with only about 394 fraud cases in the training set, QDA's per-class covariance estimates — and any model's minority-class metrics — carry wider uncertainty than the headline numbers suggest.

Findings and Conclusions

Model performance

All four models were evaluated on the same held-out test set (56,962 transactions, 98 of them fraud) at the default 0.5 threshold:

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
K-Nearest Neighbors	0.908	0.806	0.854	0.944	0.841
Logistic Regression	0.056	0.908	0.106	0.972	0.714
Linear Discriminant Analysis	0.823	0.806	0.814	0.981	0.708
Quadratic Discriminant Analysis	0.061	0.878	0.114	0.964	0.144

Model	True Positives	False Positives	False Negatives	True Negatives
K-Nearest Neighbors	79	8	19	56,856
Logistic Regression	89	1,492	9	55,372
Linear Discriminant Analysis	79	17	19	56,847
Quadratic Discriminant Analysis	86	1,327	12	55,537

K-Nearest Neighbors produced the strongest overall result: 90.8% precision and 80.6% recall (PR-AUC 0.841), meaning it caught roughly 4 out of 5 fraudulent transactions while triggering very few false alarms (8, out of 56,864 legitimate transactions in the test set). LDA was a close second and had the single highest ROC-AUC (0.981), performing well despite the imperfect normality noted during assumption testing — evidence that LDA's linear decision boundary is fairly robust on this dataset even when its formal assumptions are not fully met. Logistic Regression and QDA both achieved high recall (90.8% and 87.8% respectively) but at a steep cost in precision (5.6% and 6.1%) — each flagged well over a thousand legitimate transactions as fraud, which would overwhelm a manual review team and is not operationally practical as configured.

Threshold tuning and business impact

Using KNN (the best PR-AUC model), we swept its decision threshold and converted each point on the precision-recall curve into an estimated dollar impact: fraud dollars avoided (valued at the average fraudulent transaction amount, \$122.21) minus the labor cost of investigating each flagged transaction (a placeholder of \$5.00 per review — replace with your organization's actual cost before finalizing).

The optimal threshold identified was 0.159 (well below the default 0.5), at which the model caught 87 of 98 fraud cases in the test set (88.8% recall) while flagging 64 legitimate transactions for review — an estimated net benefit of \$10,312 on the test set alone.

Scaling this result to the full 284,807-transaction, 492-fraud dataset the test set was drawn from (the test set is a stratified 20% sample, so multiplying by ~5.02x is appropriate) implies a net benefit on the order of \$50,000–\$52,000 across the full two-day window covered by this data. Treating that two-day pattern as roughly representative and extrapolating across a full year (approximately 182 repetitions of a two-day

cycle) suggests an order-of-magnitude annual benefit in the high five to low six figures — we present this as an illustrative, back-of-envelope estimate, not a forecast: it rests on one historical two-day snapshot, an unverified \$5 review-cost placeholder, and the assumption that fraud volume and patterns stay roughly stable over time. All three should be revisited with real operational data before this number is used for planning.

Practicality for business use

KNN's inference cost scales with the size of the reference dataset, since each prediction requires computing distance to training points — a real concern for real-time scoring against a training set of hundreds of thousands of transactions. LDA, by contrast, reduces to a single linear equation at inference time and is effectively free to score in real time. This is a meaningful deployment trade-off: the model with the best offline PR-AUC (KNN) is not automatically the easiest model to serve in production, which we address directly in the Deployment Strategy below.

Model Deployment Strategy

- **Real-time path:** deploy LDA (or the tuned Logistic Regression model) as the primary real-time scorer for every transaction, since both reduce to a single fast linear computation at inference time and can meet sub-100ms scoring requirements at the point of sale.
- **Batch/offline path:** run KNN (or an approximate-nearest-neighbor variant, e.g., via FAISS, to control inference cost at scale) on a rolling batch basis — for example, hourly — to re-score recent transactions and catch cases the real-time model may have missed, routing flagged items to the fraud review queue.
- **Serialization and integration:** persist the trained model and the fitted StandardScaler together (e.g., via joblib) so the exact training-time transform is replayed at inference time; expose scoring through a lightweight internal API called by the transaction-processing pipeline.
- **Threshold as a managed parameter:** treat the decision threshold as a configurable business parameter owned jointly by data science and fraud operations, revisited whenever review capacity or the true cost of missed fraud changes — not hard-coded into the model.
- **Monitoring and retraining:** track live precision/recall against confirmed outcomes, watch for feature and score drift (fraud patterns evolve, and this dataset covers only two days of 2013 behavior), and establish a retraining cadence (e.g., monthly or upon detected drift) rather than treating the model as static.

Lessons Learned and Recommendations

Next steps

- Test ensemble methods (Random Forest, XGBoost/LightGBM) and a simple feedforward neural network, both commonly strong performers on this exact dataset in published work, as additional points of comparison beyond the four models here.
- Explore resampling techniques (SMOTE, ADASYN) as an alternative to class weighting, and compare their effect on precision at a fixed recall level.
- Replace the \$5.00 review-cost placeholder with actual fraud-operations cost data, and re-run the threshold optimization — the recommended operating point is highly sensitive to this number.
- Extend threshold tuning and net-benefit analysis to each of the four models, not just the PR-AUC leader, since operational constraints (e.g., review team capacity) may favor a different trade-off than the one that maximizes raw dollar benefit.

Additional data that could add value

- Merchant category codes, device/IP fingerprinting, or geolocation data would likely improve precision meaningfully, but are unavailable in this anonymized dataset by design.
- A longer observation window (this dataset covers only 48 hours) would help validate whether fraud patterns — and model performance — hold up over time, across days of the week, and across seasonal spending patterns.

References

Kaggle Dataset: Machine Learning Group – ULB. “Credit Card Fraud Detection.”
<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). “Calibrating Probability with Undersampling for Unbalanced Classification.” IEEE Symposium Series on Computational Intelligence (SSCI).